

C++ Pointer Programming Manual

Practical Implementation Guide & Definition Catalog using C++ Standard Library & Namespaces

Program 1: Address and Content Fundamentals

POINTER1.CPP

Demonstrates variable addresses, the address-of operator (&), and dereferencing (*&) to modify memory contents directly.

```
#include <iostream>
using namespace std;

int main() {
    int i = 10, j = 20, k = 30;
    char ch1 = 'C', ch2 = 'Z';
    float ans = 5.253f;

    cout << "Value of i is " << i << " stored at " << &i << "\n";
    cout << "Value of j is " << j << " stored at " << &j << "\n";
    cout << "Value of k is " << k << " stored at " << &k << "\n";
    cout << "Value of ch1 is " << ch1 << " stored at " << (void*)&ch1 << "\n";
    cout << "Value of ans is " << ans << " stored at " << &ans << "\n";

    // Indirect modifications via dereferenced addresses
    *&i = *&i * 10;
    *&ch1 = *&ch1 + 1;
    *&ans = 10.235f;

    cout << "\nAfter modification:\n";
    cout << "Value of i is " << *i << " stored at " << &i << "\n";
    cout << "Value of ch1 is " << *&ch1 << " stored at " << (void*)&ch1 << "\n";
    cout << "Value of ans is " << *&ans << " stored at " << &ans << "\n";
    return 0;
}
```

Key Takeaway: &x yields memory location; *&x dereferences that location to read or update the stored value.

Program 2: Pointer Variables and Manipulation

POINTER2.CPP

Explains pointer declaration, storing a variable's address in a pointer, and both direct and indirect value modifications.

```
#include <iostream>
using namespace std;

int main() {
    int i = 10;
    int* intpt = &i; // Stores address of i

    cout << "The Content of i is " << i << " and stored at " << &i << "\n";
    cout << "The Value of intpt is " << intpt << " and stored at " << &intpt << "\n";
    cout << "The Content of intpt is " << *intpt << " and stored at " << intpt << "\n";

    i = 20; // Direct change
    cout << "\nDirect change: *intpt = " << *intpt << " at " << intpt << "\n";

    *intpt = *intpt + 10; // Indirect change
    cout << "\nIndirect change: *intpt = " << *intpt << " at " << intpt << "\n";
    return 0;
}
```

Program 3: Dynamic Allocation & Pointer Traversal

POINTER3.CPP

Allocates heap memory dynamically using `new[]`, captures inputs into contiguous memory offsets, and displays each value.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "How many values are there? ";
    cin >> n;

    // Safely allocate contiguous block of heap memory
    int* ptr = new int[n];
    cout << "Initial value of ptr: " << ptr << " and address: " << &ptr << "\n";

    for (int i = 0; i < n; i++) {
        cout << "Enter ptr value [" << i << "] at address " << (ptr + i) << ": ";
        cin >> *(ptr + i);
    }

    cout << "\nDisplay the value on Screen for ptr:\n";
    for (int i = 0; i < n; ++i) {
        cout << "The value is: " << *(ptr + i) << " and address: " << (ptr + i) << "\n";
    }

    delete[] ptr; // Clean up memory
    return 0;
}
```

Program 4: Character Pointer Traversal

POINTER5.CPP

Reads a text line into a bounded character array and iterates through each character using pointer increment arithmetic.

```
#include <iostream>
using namespace std;

int main() {
    char buffer[100];
    cout << "Enter the string: ";
    cin.getline(buffer, 100);

    char* ptr = buffer;
    cout << "The string is: " << buffer << "\n";
    cout << "Starting content: " << *ptr << " at address: " << (void*)ptr << "\n\n";

    while (*ptr != '\0') {
        cout << "content is " << *ptr << " | Address: " << (void*)ptr << "\n";
        ptr++;
    }
    return 0;
}
```

Note: Casting `char*` to `(void*)` forces `cout` to print the memory address rather than printing it as a C-string.

Program 5: Forward Input and Reverse Output with Pointers

POINTER6.CPP

Accepts 5 integer values sequentially by advancing a pointer, then displays them in reverse order using pointer decrements.

```
#include <iostream>
using namespace std;

int main() {
    int arr[5];
    int* ptr = arr;

    cout << "Initial ptr value: " << ptr << " stored at: " << &ptr << "\n";

    for (int i = 0; i < 5; i++) {
        cout << "Enter value at " << ptr << ": ";
        cin >> *ptr;
        ptr++;
    }

    cout << "\nDisplaying in reverse order:\n";
    for (int i = 0; i < 5; i++) {
        ptr--;
        cout << "Value: " << *ptr << " and address: " << ptr << "\n";
    }
    return 0;
}
```

Program 6: Standard String Library Functions with Pointers

PTLIBFUN.CPP

Illustrates pointer-based string handling with strlen, strcpy, strcat, and strcmp using mutable memory buffers.

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char str1[100] = "Bhavnagar ";
    char str2[50] = "University";
    char str3[100];

    char* ptr1 = str1;
    char* ptr2 = str2;
    char* ptr3 = str3;

    cout << "Length of ptr1: " << strlen(ptr1) << " at " << (void*)ptr1 << "\n";
    cout << "Length of ptr2: " << strlen(ptr2) << " at " << (void*)ptr2 << "\n";

    strcpy(ptr3, ptr2);
    cout << "After strcpy, ptr3: " << ptr3 << "\n";

    strcat(ptr1, "Ram");
    cout << "After strcat(ptr1, 'Ram'): " << ptr1 << "\n";

    if (strcmp(ptr1, ptr3) > 0)
        cout << "ptr1 string is lexicographically greater than ptr3\n";
    else
        cout << "ptr3 string is greater than or equal to ptr1\n";

    return 0;
}
```

Program 7: Finding Array Maximum using Pointers

PTMAXFN.CPP

Scans through an array using a pointer cursor and condition checks to identify the maximum numeric element.

```
#include <iostream>
using namespace std;

int main() {
    float a[6] = {6.2f, 62.23f, 1.0f, 0.0f, -55.24f, 21.09f};
    float* ptr = a;
    float max = *ptr;

    cout << "Base Address of a is: " << a << "\n";
    for (int i = 0; i < 6; i++) {
        cout << "Content of a[" << i << "] = " << *ptr << " Address: " << ptr << "\n";
        if (*ptr > max) {
            max = *ptr;
        }
        ptr++;
    }
    cout << "\nMaximum is " << max << "\n";
    return 0;
}
```

Program 8: Array and Pointer Equivalence

POINTER9.CPP

Proves equivalence between subscript notation `a[i]`, pointer arithmetic `*(ptr+i)`, and base offset `*(a+i)`.

```
#include <iostream>
using namespace std;

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int* ptr = a;

    cout << "1. Display using Array Subscript a[i]:\n";
    for (int i = 0; i < 5; i++)
        cout << "a[" << i << "] = " << a[i] << " at " << &a[i] << "\n";

    cout << "\n2. Display using Pointer Arithmetic *(ptr + i):\n";
    for (int i = 0; i < 5; i++)
        cout << "*(ptr+" << i << ") = " << *(ptr + i) << " at " << (ptr + i) << "\n";

    cout << "\n3. Display using Array Name Offset *(a + i):\n";
    for (int i = 0; i < 5; i++)
        cout << "*(a+" << i << ") = " << *(a + i) << " at " << (a + i) << "\n";

    return 0;
}
```